Practical:09	Implementation of Prim's algorithm using greedy method.
__/__ /2026	

Objective:

- To implement Prim's algorithm to find the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph.
- To understand how the greedy method picks the cheapest edge that connects a new vertex to the tree.
- To analyze the time and space complexity of Prim's algorithm.

Program:

```
# Prim's Minimum Spanning Tree (adjacency matrix)
INF = float('inf')

graph = [
    [0, 4, 3, 0, 0, 0],
    [4, 0, 1, 2, 0, 0],
    [3, 1, 0, 4, 0, 0],
    [0, 2, 4, 0, 2, 7],
    [0, 0, 0, 2, 0, 6],
    [0, 0, 0, 7, 6, 0]
]

n = len(graph)
key = [INF] * n # cheapest edge weight connecting each vertex to the tree
parent = [-1] * n # tree vertex that the cheapest edge comes from
in_mst = [False] * n
key[0] = 0 # start from vertex 0
total = 0

print("PRIM'S ALGORITHM")
for step in range(n):
    # pick the vertex outside the tree with the smallest key
    u = -1
    for v in range(n):
```

```
        if not in_mst[v] and (u == -1 or key[v] < key[u]):
            u = v
    if key[u] == INF:
        raise ValueError("Graph is disconnected")
    in_mst[u] = True
    total += key[u]
    if parent[u] == -1:
        print(f"Step {step}: start at vertex {u}")
    else:
        print(f"Step {step}: add vertex {u} using edge {parent[u]}-{u} (weight {key[u]})")
    # update keys of the neighbours of u
    for v in range(n):
        if graph[u][v] > 0 and not in_mst[v] and graph[u][v] < key[v]:
            key[v] = graph[u][v]
            parent[v] = u
print("Edges in the MST:")
for v in range(1, n):
    print(f" {parent[v]} - {v} : {graph[parent[v]][v]}")
print("Total cost of MST =", total)
```

Output:

```
PRIM'S ALGORITHM
Step 0: start at vertex 0
Step 1: add vertex 2 using edge 0-2 (weight 3)
Step 2: add vertex 1 using edge 2-1 (weight 1)
Step 3: add vertex 3 using edge 1-3 (weight 2)
Step 4: add vertex 4 using edge 3-4 (weight 2)
Step 5: add vertex 5 using edge 4-5 (weight 6)
Edges in the MST:
  2 - 1 : 1
  0 - 2 : 3
  1 - 3 : 2
  3 - 4 : 2
  4 - 5 : 6
Total cost of MST = 14
```

Analysis:

a) Problem Analysis

A spanning tree of a graph connects all V vertices using exactly V - 1 edges and has no cycle. A Minimum Spanning Tree is the spanning tree whose total edge weight is the smallest. Typical uses are laying cables, pipelines or roads at minimum cost.

b) Greedy Approach

Prim's algorithm grows one tree, starting from any vertex. At every step it looks at all edges that connect a vertex inside the tree to a vertex outside it, and adds the cheapest one. This is safe because the cheapest edge crossing any cut of the graph always belongs to some MST (the cut property).

To find the cheapest edge quickly, the program keeps a key value for every vertex outside the tree: the weight of the cheapest edge that connects it to the tree. The vertex with the smallest key is added next.

c) Example Analysis

Consider the graph with 6 vertices (0 to 5) and these edges:

Edge	Weight
0 - 1	4
0 - 2	3
1 - 2	1
1 - 3	2
2 - 3	4
3 - 4	2
3 - 5	7
4 - 5	6

Trace of the algorithm (start at vertex 0):

Step	Vertex added	Edge used	Cost so far	Keys after update
0	0	-	0	[-, 4, 3, inf, inf, inf]
1	2	0-2 (3)	3	[-, 1, -, 4, inf, inf]
2	1	2-1 (1)	4	[-, -, -, 2, inf, inf]
3	3	1-3 (2)	6	[-, -, -, -, 2, 7]
4	4	3-4 (2)	8	[-, -, -, -, -, 6]
5	5	4-5 (6)	14	[-, -, -, -, -, -]

The tree has 5 edges (V - 1) and no cycle. Total cost of the MST = 14.

d) Algorithm Analysis

Step 1: Set key[0] = 0 and the key of every other vertex = infinity. No vertex is in the tree yet.

Step 2: Pick the vertex u that is not in the tree and has the smallest key.

Step 3: Add u to the tree and add key[u] to the total cost.

Step 4: For every neighbour v of u that is not in the tree, if the edge weight is less than key[v], update key[v] and parent[v].

Step 5: Repeat steps 2 to 4 until all V vertices are in the tree. If the smallest key is infinity, the graph is disconnected.

e) Time Complexity

The outer loop runs V times. In each round, finding the smallest key scans V vertices, and updating the neighbours scans one matrix row of V entries.

Therefore:

Time Complexity = $O(V^2)$

With an adjacency list and a min-heap (priority queue), the time becomes O(E log V), which is better for sparse graphs.

f) Space Complexity

The adjacency matrix needs $O(V^2)$ space. The key, parent and in_mst arrays need only O(V) extra space.

Therefore:

Space Complexity = $O(V^2)$ for the matrix, and O(V) auxiliary

Conclusion:

Prim's algorithm was successfully implemented in Python using the greedy method. For the given graph it selected 5 edges with a total MST cost of 14. The adjacency-matrix version runs in $O(V^2)$ time, and a heap-based version gives O(E log V).